

URL Reserved Characters Reference

Characters You MUST Encode in Query Strings

Character	Encoded	Why?	Example
Space	%20 or +	Breaks URL structure	hello world → hello%20world
/ (Forward Slash)	%2F	Confused with directory paths	date/time → date%2Ftime
? (Question Mark)	%3F	Marks start of query params	what? → what%3F
& (Ampersand)	%26	Separates query parameters	A&B → A%26B
# (Hash)	%23	Client-side anchor/fragment	#tag → %23tag
= (Equals)	%3D	Separates key from value	a=b → a%3Db
+ (Plus)	%2B	Often decoded as space	1+1 → 1%2B1
% (Percent)	%25	Start of encoding sequence	50% → 50%25

Complete Reserved Characters Table

RFC 3986 Reserved Characters

Character	Encoded	Name	Usage
!	%21	Exclamation	Sub-delimiters
#	%23	Hash/Pound	Fragment identifier

Character	Encoded	Name	Usage
\$	%24	Dollar	Sub-delimiters
&	%26	Ampersand	Query separator
'	%27	Apostrophe	Sub-delimiters
(	%28	Open Parenthesis	Sub-delimiters
)	%29	Close Parenthesis	Sub-delimiters
*	%2A	Asterisk	Sub-delimiters
+	%2B	Plus	Encoded space (alt)
,	%2C	Comma	Sub-delimiters
/	%2F	Forward Slash	Path separator
:	%3A	Colon	Scheme/port delimiter
;	%3B	Semicolon	Sub-delimiters
=	%3D	Equals	Key-value separator
?	%3F	Question Mark	Query string start
@	%40	At Sign	Authority delimiter
[	%5B	Open Bracket	IPv6 literals
]	%5D	Close Bracket	IPv6 literals

When to Encode vs. Not Encode

✓ Always Encode in Query Parameters

None

```
?search=hello%20world&filter=price%3E100
```

❌ Don't Encode in URL Path (Some Characters)

None

`https://example.com/api/users/123`

✅ Forward slash is OK in path

⚠️ Special Case: Plus (+) vs Space (%20)

None

Query: `?q=hello+world` ← Plus decoded as space (legacy)

Query: `?q=hello%20world` ← Modern standard

Recommendation: Always use `%20` for consistency

JavaScript Implementation

Encoding Functions

`encodeURIComponent()` - For Query Parameters (RECOMMENDED)

JavaScript

```
const searchTerm = "price > $100";
const encoded = encodeURIComponent(searchTerm);
// → "price%20%3E%20%24100"

const url = `https://api.com/search?q=${encoded}`;
// → https://api.com/search?q=price%20%3E%20%24100
```

Encodes: Everything except `A-Z a-z 0-9 - _ . ! ~ * ' ( )`

`encodeURI()` - For Full URLs (AVOID for values)

JavaScript

```
const url = "https://example.com/search?q=hello world";
const encoded = encodeURI(url);
// → "https://example.com/search?q=hello%20world"
```

PROBLEM: Doesn't encode `& = / ? #` (breaks query values)

JavaScript

```
// ❌ WRONG for query values
const value = "a=b&c=d";
encodeURIComponent(value); // → "a=b&c=d" (not encoded!)

// ✅ CORRECT
encodeURIComponent(value); // → "a%3Db%26c%3Dd"
```

escape() - DEPRECATED, DO NOT USE

JavaScript

```
// ❌ NEVER USE (deprecated)
escape("hello world"); // Non-standard encoding
```

Decoding Functions

decodeURIComponent() - Decode Query Parameters

JavaScript

```
const encoded = "price%20%3E%20%24100";
const decoded = decodeURIComponent(encoded);
// → "price > $100"
```

decodeURI() - Decode Full URLs

JavaScript

```
const encoded = "https://example.com/search?q=hello%20world";
const decoded = decodeURI(encoded);
// → "https://example.com/search?q=hello world"
```

Common Mistakes

❌ Mistake 1: Double Encoding

JavaScript

```
const value = "hello world";
```

```
const once = encodeURIComponent(value); // "hello%20world"
const twice = encodeURIComponent(once); // "hello%2520world" ❌
```

Result: hello%2520world instead of hello%20world

❌ Mistake 2: Using encodeURIComponent() for Parameters

```
JavaScript

const key = "redirect_url";
const value = "https://example.com/page?id=123";

// ❌ WRONG: encodeURIComponent doesn't encode & = ?
const wrong = `${key}=${encodeURIComponent(value)}`;
// → "redirect_url=https://example.com/page?id=123"
// The ? and = break the query string!

// ✅ CORRECT
const right = `${key}=${encodeURIComponent(value)}`;
// → "redirect_url=https%3A%2F%2Fexample.com%2Fpage%3Fid%3D123"
```

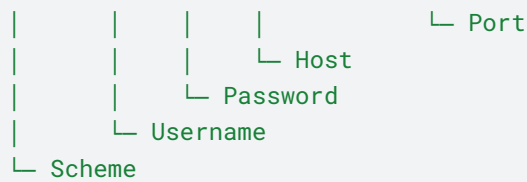
✗ Mistake 3: Not Encoding at All

```
JavaScript
// ❌ WRONG
const url = `https://api.com/search?q=${searchTerm}`;
// If searchTerm = "a&b", you get: ?q=a&b (broken query string)

// ✅ CORRECT
const url = `https://api.com/search?q=${encodeURIComponent(searchTerm)}`;
```

URL Components Breakdown

```
None
https://user:pass@example.com:8080/path/to/page?key=value&foo=bar#section
| | | | | | | |
| | | | | | | |
Fragment (not sent to server)
| | | | | | | |
| | | | | | | |
| | | | | | | |
```



What to Encode in Each Part

Component	Encoding Required	Function
Scheme	None	<code>https</code>
Username	Yes (if special chars)	<code>encodeURIComponent()</code>
Password	Yes (if special chars)	<code>encodeURIComponent()</code>
Host	No (IDN for internationalized)	Native or punycode
Port	No	Numbers only
Path	Partial	Some chars allowed
Query	Yes	<code>encodeURIComponent()</code> for values
Fragment	Yes	<code>encodeURIComponent()</code>

Other Programming Languages

Python

```
Python
from urllib.parse import quote, quote_plus, unquote

# Encode (equivalent to encodeURIComponent)
encoded = quote("price > $100", safe='')
# → "price%20%3E%20%24100"

# Encode with + for spaces (legacy)
encoded_plus = quote_plus("price > $100")
# → "price+%3E+%24100"
```

```
# Decode
decoded = unquote("price%20%3E%20%24100")
# → "price > $100"
```

PHP

```
PHP

// Encode (equivalent to encodeURIComponent)
$encoded = rawurlencode("price > $100");
// → "price%20%3E%20%24100"

// Encode with + for spaces (legacy form data)
$encoded_plus = urlencode("price > $100");
// → "price+%3E+%24100"

// Decode
$decoded = urldecode("price%20%3E%20%24100");
// → "price > $100"
```

Java

```
Java

import java.net.URLEncoder;
import java.net.URLDecoder;

// Encode
String encoded = URLEncoder.encode("price > $100", "UTF-8");
// → "price+%3E+%24100" (uses + for spaces)

// Decode
String decoded = URLDecoder.decode(encoded, "UTF-8");
// → "price > $100"
```

SEO Considerations

URL Structure Best Practices

None

- ✓ GOOD: <https://example.com/blog/how-to-encode-urls>
- ✗ BAD: <https://example.com/blog/post.php?id=123&cat=tech&page=5>

Why?

- Clean URLs are more user-friendly
- Better for SEO (keywords in path)
- Easier to share and remember

Handling Special Characters in Slugs

JavaScript

```
function createSlug(title) {  
  return title  
    .toLowerCase()  
    .replace(/[^\w\s-]/g, '') // Remove special chars  
    .replace(/\s+/g, '-')    // Replace spaces with hyphens  
    .replace(/-+/g, '-')    // Replace multiple hyphens  
    .trim();  
}  
  
createSlug("How to Encode URLs? (A Guide)");  
// → "how-to-encode-urls-a-guide"
```

International Characters (UTF-8)

JavaScript

```
// UTF-8 characters are automatically encoded  
const chinese = "搜索";  
encodeURIComponent(chinese);  
// → "%E6%90%9C%E7%B4%A2"  
  
// Or use readable format in path (if supported)  
const url = "https://example.com/search/搜索";  
// Browser handles encoding automatically
```

Security Implications

SQL Injection via URL Parameters

PHP

```
// ❌ VULNERABLE
$id = $_GET['id']; // Attacker sends: ?id=1' OR '1'='1
$query = "SELECT * FROM users WHERE id='$id'";
// Result: SELECT * FROM users WHERE id='1' OR '1'='1'

// ✅ SAFE (parameterized query)
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
$stmt->execute([$_GET['id']]);
```

Note: URL encoding doesn't prevent SQL injection. Always use parameterized queries.

XSS via URL Parameters

JavaScript

```
// ❌ VULNERABLE
const params = new URLSearchParams(window.location.search);
document.getElementById('message').innerHTML = params.get('msg');
// Attacker sends: ?msg=<script>alert('XSS')</script>

// ✅ SAFE
document.getElementById('message').textContent = params.get('msg');
// Or use DOMPurify to sanitize
```

Open Redirect Vulnerability

JavaScript

```
// ❌ VULNERABLE
const redirect = params.get('redirect');
window.location = redirect; // Attacker sends:
?redirect=https://malicious.com

// ✅ SAFE (validate redirect URL)
const redirect = params.get('redirect');
if (redirect.startsWith('/') || redirect.startsWith('https://trusted.com'))
{
    window.location = redirect;
}
```

Testing URLs

Manual Testing

Shell

```
# Test encoding
echo "hello world" | jq -sRr @uri
# → hello%20world

# Test full URL
curl "https://api.example.com/search?q=price%20%3E%20100"
```

Automated Testing

JavaScript

```
describe('URL Encoding', () => {
  it('encodes special characters correctly', () => {
    const input = 'price > $100 & quantity < 50';
    const encoded = encodeURIComponent(input);

    expect(encoded).toBe('price%20%3E%20%24100%20%26%20quantity%20%3C%2050');
    expect(decodeURIComponent(encoded)).toBe(input);
  });

  it('handles unicode characters', () => {
    const input = '你好世界';
    const encoded = encodeURIComponent(input);

    expect(decodeURIComponent(encoded)).toBe(input);
  });
});
```

Quick Reference Cheatsheet

None

SPACE	→ %20 or +
!	→ %21
"	→ %22
#	→ %23
\$	→ %24
%	→ %25

&	→ %26
'	→ %27
(	→ %28
)	→ %29
*	→ %2A
+	→ %2B
,	→ %2C
/	→ %2F
:	→ %3A
;	→ %3B
=	→ %3D
?	→ %3F
@	→ %40
[	→ %5B
]	→ %5D

Need SEO Help?

Fixing technical SEO issues?

Devxhub offers comprehensive technical SEO audits:

- ✓ URL structure optimization
- ✓ Page speed optimization
- ✓ Mobile-first design
- ✓ Schema markup implementation
- ✓ Core Web Vitals improvement
- ✓ International SEO (hreflang)

Our Services:

- Technical SEO audit
- Site speed optimization
- SEO-friendly architecture
- Content delivery optimization

✉ Contact us: info@devxhub.com

🌐 Visit: www.devxhub.com

☎ Call: +880 1326 506464